

PRESIDENCY UNIVERSITY

CSE208: Digital Logic Design Laboratory

Lecture 06: Multiplexer (MUX)

Data Selection • Select Lines • Logic Design
Boolean Functions • IC Verification • Experiments

Afzal Hossan
Lecturer
Department of Computer Science and Engineering
Presidency University

Learning outcomes

BY THE END OF THIS LABORATORY

01 Explain
Identify data inputs, select lines, enable, and output.

02 Analyze
Read truth tables of 2:1, 4:1, and 8:1 MUXs.

03 Derive
Obtain Boolean expressions and gate-level circuits.

04 Design
Cascade MUXs and implement Boolean functions.

05 Construct
Wire and operate the SN74LS153 safely.

06 Verify
Test expected outputs and troubleshoot errors.

Aligned with CSE208 Lab 6 and course outcome CO3.

Start from what you already know

PREREQUISITE BRIDGE: LABS 1–5

Logic gates

AND AB

OR $A + B$

NOT $\overline{A}$

Binary selection

$00_2 = 0, \quad 01_2 = 1, \quad 10_2 = 2, \quad 11_2 = 3$

Truth tables and Boolean functions

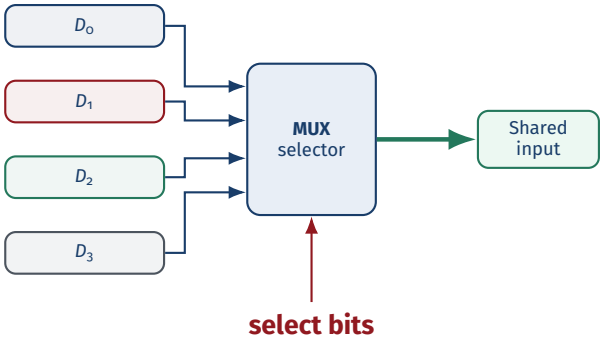
- Inputs determine a combinational output.
- No clock and no stored state are needed.

Connection to earlier labs

K-maps, adders, and subtractors are combinational circuits; a MUX is another one.

The problem: many sources, one destination

REAL-WORLD MOTIVATION



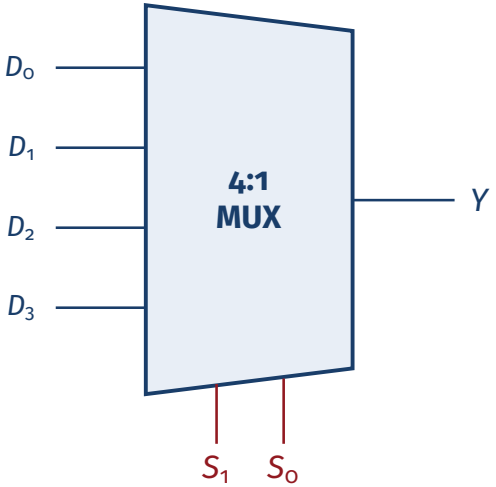
Design question

How can *one of four* digital signals use *one shared* destination?

Solution

A multiplexer routes the **selected signal** to the output.

MUX anatomy: four parts, one job



Data inputs D_0–D_3
Carry the candidate logic signals.

Select lines S_1, S_0
Choose *which* data input is passed.

Output Y
Copies the selected input's logic level.

Enable (on many ICs)
Allows or disables normal selection.

How many select lines are needed?

THE FUNDAMENTAL SIZING RULE

$$N = 2^n$$

N data inputs; n binary select lines

$n = \log_2 N$ (if N is a power of two)

Example

8 inputs need $n = 3$ select bits because $2^3 = 8$.

MUX	Inputs	Select bits
2:1	2	1
4:1	4	2
8:1	8	3
16:1	16	4

For arbitrary N

At least $\lceil \log_2 N \rceil$ bits are needed. If $N = 5$, three bits permit eight codes; three codes must be handled appropriately.

Enable is not a data select line

Active-high enable (E)

$E = 1$: device operates; $E = 0$: disabled.

Active-low enable ($\overline{G}$)

$\overline{G} = 0$: device operates; $\overline{G} = 1$: disabled.

Important

The output level when disabled is *IC-specific*. Disabled does not automatically mean high impedance.

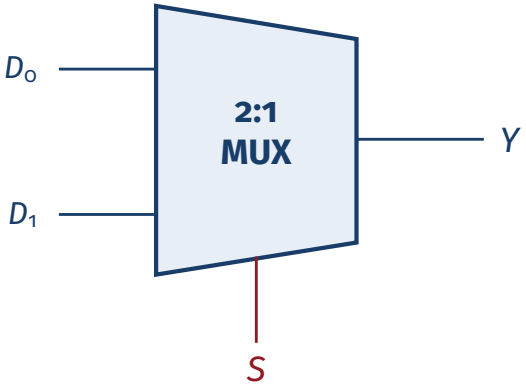
SN74LS153 example

$\overline{G}$	Output Y
0	selected D_i
1	0 (LOW)

Key idea Select bits choose the source; the enable determines whether the section is active.

Hardware behavior: Texas Instruments SN74LS153 function table.

2:1 MUX — the smallest selector



S	Selected	Y
0	D_0	D_0
1	D_1	D_1

Try it

Set $D_0 = 0$, $D_1 = 1$. Output is 0 for $S = 0$ and 1 for $S = 1$.

Key idea S selects an *input*, not a numerical value to place at the output.

Derive the 2:1 MUX Boolean expression

EXPLAIN EACH PRODUCT TERM

Case 1: $S = 0$
$\overline{S} = 1$; select D_0.
Contribution: $\overline{S}D_0$

Case 2: $S = 1$
$S = 1$; select D_1.
Contribution: SD_1

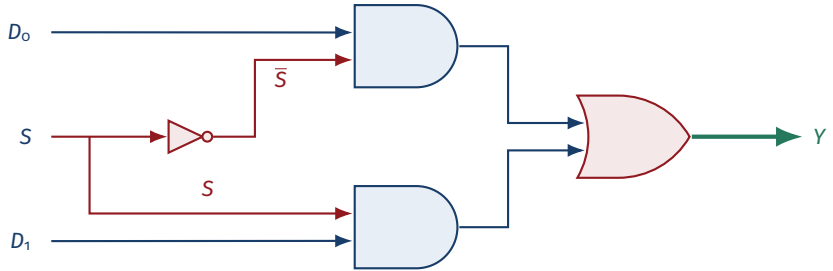
Combine cases using OR

$$\boxed{Y = \overline{S}D_0 + SD_1}$$

Check
At $S = 0$, $Y = D_0$. At $S = 1$, $Y = D_1$.

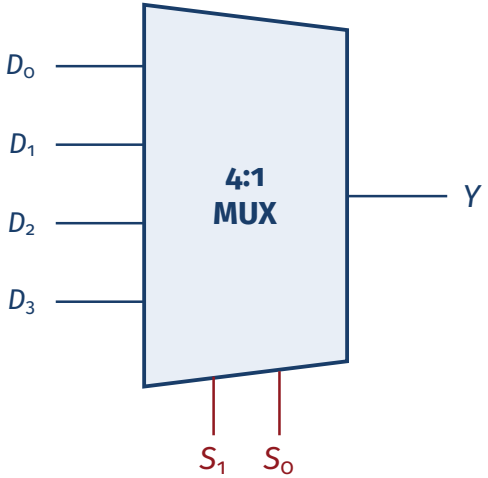
2:1 MUX built using logic gates

ONE NOT, TWO ANDS, ONE OR



$Y = (\overline{S} \wedge D_0) \vee (S \wedge D_1)$ Exactly the same equation as the previous slide.

4:1 MUX — one of four signals



Four data inputs
D_0, D_1, D_2, D_3

Two select bits
S_1S_0 is a two-bit binary index.

One output
Y copies the selected D_i.

4:1 selection table

S_1	S_0	S_1S_0	Y
0	0	$00_2 = 0$	D_0
0	1	$01_2 = 1$	D_1
1	0	$10_2 = 2$	D_2
1	1	$11_2 = 3$	D_3

Binary select value = chosen input index.

Example

Let $(D_0, D_1, D_2, D_3) = (1, 0, 1, 0)$.

If $S_1S_0 = 10$, then $Y = D_2 = 1$.

If $S_1S_0 = 11$, then $Y = D_3 = 0$.

Common error

D_2 is a *signal name*, not the constant "2."

Deriving the 4:1 MUX expression

EACH SELECT COMBINATION ENABLES ONE TERM

S_1S_0	Active minterm	Data
00	$\overline{S}_1\overline{S}_0$	D_0
01	$\overline{S}_1S_0$	D_1
10	$S_1\overline{S}_0$	D_2
11	S_1S_0	D_3

Product terms

$T_0 = \overline{S}_1\overline{S}_0D_0$

$T_1 = \overline{S}_1S_0D_1$

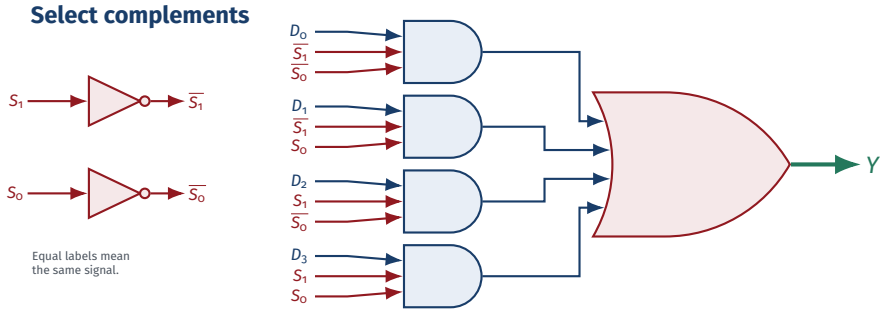
$T_2 = S_1\overline{S}_0D_2$

$T_3 = S_1S_0D_3$

$$\boxed{Y = \overline{S}_1\overline{S}_0D_0 + \overline{S}_1S_0D_1 + S_1\overline{S}_0D_2 + S_1S_0D_3}$$

Gate-Level Implementation of a 4:1 Multiplexer

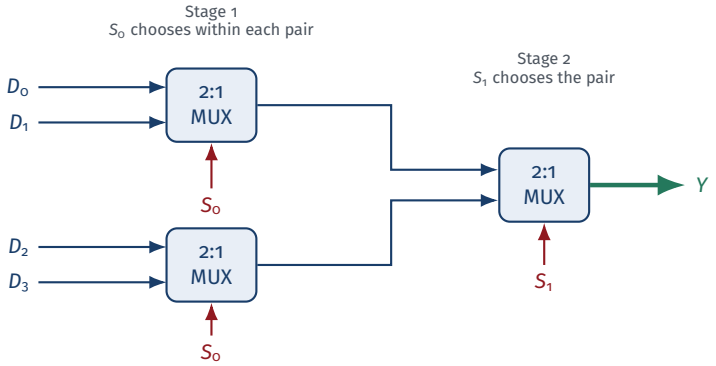
TWO NOT GATES • FOUR 3-INPUT AND GATES • ONE 4-INPUT OR GATE



$$Y = \overline{S_1}\overline{S_0}D_0 + \overline{S_1}S_0D_1 + S_1\overline{S_0}D_2 + S_1S_0D_3$$

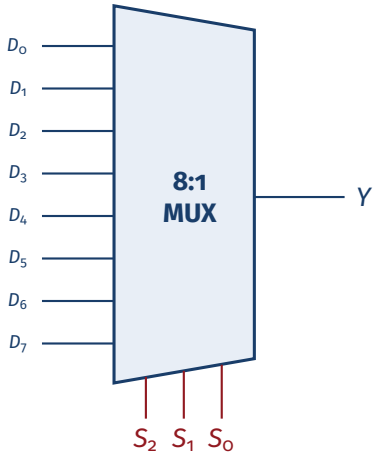
A 4:1 MUX from three 2:1 MUXs

HIERARCHICAL CIRCUIT CONSTRUCTION



$S_1S_0 = 10$: $S_1 = 1$ chooses the D_2/D_3 pair, and $S_0 = 0$ selects D_2.

8:1 MUX: selecting one of eight inputs



$S_2S_1S_0$	Y	$S_2S_1S_0$	Y
000	D_0	100	D_4
001	D_1	101	D_5
010	D_2	110	D_6
011	D_3	111	D_7

Why three bits?

Three bits have $2^3 = 8$ possible codes.

Read select bits as a binary address

THREE EXAMPLES TO TRACE

Case A

010_2

$= 2_{10}$

$Y = D_2$

Case B

101_2

$= 5_{10}$

$Y = D_5$

Case C

111_2

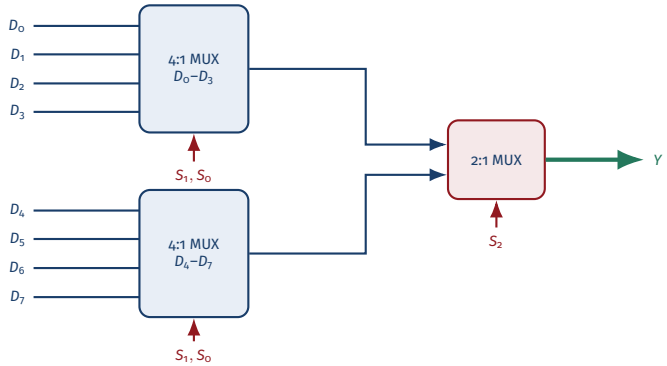
$= 7_{10}$

$Y = D_7$

Quick check What if $D_5 = 0$ but $S_2S_1S_0 = 101$? **Answer:** $Y = 0$.

Build an 8:1 MUX by cascading smaller MUXs

TWO 4:1 SECTIONS + ONE 2:1 SELECTOR



S_1, S_0 select within each group; S_2 chooses the group.

Why can a MUX implement a Boolean function?

START WITH THE PHYSICAL MEANING OF A SELECT LINE

A 2:1 MUX makes one choice

S	Selected input	Output
0	D_0	$Y = D_0$
1	D_1	$Y = D_1$

$$Y = \overline{S}D_0 + SD_1$$

Apply the same idea to any function F

When $S = 0$, the desired output is F evaluated with $S = 0$.

When $S = 1$, the desired output is F evaluated with $S = 1$.

Therefore, wire $D_0 = F\big|_{S=0}$ $D_1 = F\big|_{S=1}$ $S =$ chosen variable.

The notation $F\big|_{S=0}$ is called a *cofactor*: substitute $S = 0$, then simplify the remaining expression.

Where does Shannon expansion come from?

DERIVE THE IDENTITY INSTEAD OF MEMORIZING A FORMULA

Every Boolean select variable satisfies $\overline{S} + S = 1$.

$$\begin{aligned} F &= F \cdot 1 \\ &= F(\overline{S} + S) && \text{replace } 1 \text{ with } \overline{S} + S \\ &= \overline{S}F + SF && \text{distributive law.} \end{aligned}$$

When $S = 0$

$\overline{S} = 1$, so $\overline{S}F$ matters.

In this term F is evaluated with $S = 0$:

$$\overline{S}F = \overline{S}\,F\big|_{S=0}.$$

When $S = 1$

$S = 1$, so SF matters.

In this term F is evaluated with $S = 1$:

$$SF = S\,F\big|_{S=1}.$$

Shannon expansion $\boxed{F = \overline{S}\,F\big|_{S=0} + S\,F\big|_{S=1}}$

Technique A: find cofactors from an expression

SUBSTITUTE FIRST, SIMPLIFY SECOND, CONNECT THIRD

Given $F(A, B, C) = \overline{A}(B + C) + A(\overline{B}C)$ **Choose select** $S = A$.

Case 1: set $A = 0$

$D_0 = F(0, B, C)$

$= \overline{0}(B + C) + 0(\overline{B}C)$

$= 1(B + C) + 0$

$= \boxed{B + C}$.

Case 2: set $A = 1$

$D_1 = F(1, B, C)$

$= \overline{1}(B + C) + 1(\overline{B}C)$

$= 0 + 1(\overline{B}C)$

$= \boxed{\overline{B}C}$.

Connect: $S = A$; $D_0 = B + C$ (OR gate); $D_1 = \overline{B}C$ (NOT + AND).

Technique B: find cofactors from a truth table

USEFUL WHEN THE FUNCTION IS GIVEN AS A LIST OF MINTERMS

Given $F(A, B, C) = \Sigma m(1, 2, 6, 7)$ **Select** $S = A$. (A is the MSB.)

Full function truth table

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Pair rows that have the same B, C

B	C	$D_0 = F(0, B, C)$	$D_1 = F(1, B, C)$
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

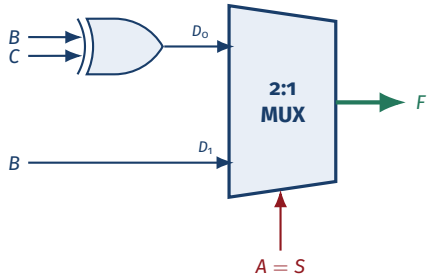
$D_0 = \overline{B}C + B\overline{C} = B \oplus C$

$D_1 = B$.

Reading the result: if $A = 0$, output $B \oplus C$; if $A = 1$, output B.

Turn the cofactors into actual MUX connections

THE MUX SELECTS; EXTRA GATES COMPUTE ITS DATA SIGNALS



Exact wiring

Select: $S = A$

Input 0: $D_0 = B \oplus C$

Input 1: $D_1 = B$

Output equation

$F = \overline{A}(B \oplus C) + AB$

Check both select states:

$A = 0 \Rightarrow F = B \oplus C; \qquad A = 1 \Rightarrow F = B.$

How should you choose the select variable?

DIFFERENT VALID CHOICES CAN REQUIRE DIFFERENT EXTRA GATES

Example $F(A, B, C) = \overline{A}B + AC$ Choose S to make D_0 and D_1 easy to construct.

S	$D_0 = F\|_{S=0}$	$D_1 = F\|_{S=1}$	**Extra data logic**
A	B	C	**None (best here)**
B	AC	$\overline{A} + C$	AND, NOT, OR
C	$\overline{A}B$	$A + B$	NOT, AND, OR

All three choices are correct.
Each produces the same original truth table.

But $S = A$ is simplest here.
Connect $D_0 = B$ and $D_1 = C$ directly.

Shannon-to-MUX design: a reusable procedure

FOLLOW THESE STEPS FOR EVERY BOOLEAN FUNCTION

Step 1: Choose a select variable S.
Step 2: Compute $D_0 = F|_{S=0}$.
Step 3: Compute $D_1 = F|_{S=1}$.
Step 4: Simplify both cofactors.
Step 5: Connect and verify.

Avoid these common errors

- Do not leave S inside D_0 or D_1 after substitution.
- Do not swap the $S = 0$ and $S = 1$ inputs.
- Do not assume D_0, D_1 must be constants.
- Check the function for both select states.

Final sanity check $S = 0 \Rightarrow Y = D_0 = F|_{S=0}$, $S = 1 \Rightarrow Y = D_1 = F|_{S=1}$.

Boolean functions: three sizes, nine core cases

PROGRESS FROM SIMPLE CONSTANTS TO LOGIC AT DATA INPUTS

2:1 MUX
1 select line
Functions of 1, 2, 3 variables

4:1 MUX
2 select lines
Functions of 2, 3, 4 variables

8:1 MUX
3 select lines
Functions of 3, 4, 5 variables

Design rule: for a 2^k:1 MUX, choose k variables for the select pins. For each select combination, evaluate and simplify the remaining-variable function, then connect it to the corresponding data pin.

Notation throughout: $\Sigma m(\ldots)$ assumes standard binary order (A is the most significant variable). D_i is a *data pin*, not the variable D. Extra logic may be required to generate a data-pin expression.

2:1 MUX example A: one-variable function

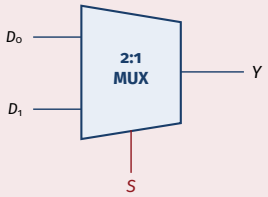
IMPLEMENT A NOT FUNCTION USING ONLY A MUX AND CONSTANTS

Target: $F(A) = \overline{A}$

A	$F = \overline{A}$	MUX action
0	1	select D_0
1	0	select D_1

Direct connections

$S = A, \quad D_0 = 1, \quad D_1 = 0$



$D_0 \leftarrow 1$ (HIGH)
$D_1 \leftarrow 0$ (LOW)
$S \leftarrow A$; output $Y = F$.

$$Y = \overline{S}D_0 + SD_1 = \overline{A}(1) + A(0) = \overline{A}.$$

2:1 MUX example B: two-variable function

USE THE REMAINING VARIABLE AND ITS COMPLEMENT AT THE DATA PINS

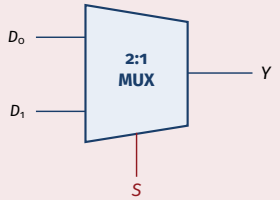
Target: $F(A, B) = A \oplus B = \Sigma m(1, 2)$

Choose: $S = A$. Fix A and examine B.

A	$F(B=0)$	$F(B=1)$	Data function
0	0	1	$D_0 = B$
1	1	0	$D_1 = \overline{B}$

$$\boxed{S = A, \quad D_0 = B, \quad D_1 = \overline{B}}$$

A NOT gate produces $\overline{B}$; the MUX performs selection.



$$D_0 \leftarrow B$$
$$D_1 \leftarrow \overline{B}$$
$$S \leftarrow A$$

Verify: $A = 0 \Rightarrow F = B$; $\quad A = 1 \Rightarrow F = \overline{B}$.

2:1 MUX bridge: direct matching with three variables

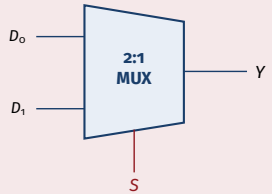
BEFORE DIFFICULT TRUTH TABLES, RECOGNIZE AN EXISTING MUX PATTERN

Given: $F(A, B, C) = \overline{A}B + AC$

MUX equation: $Y = \overline{S}D_0 + SD_1$

Match the barred selector with $\overline{A}$ and the unbarred selector with A.

$$\boxed{S = A, \quad D_0 = B, \quad D_1 = C}$$



$S \leftarrow A$
$D_0 \leftarrow B$
$D_1 \leftarrow C$

Why it works: $A = 0 \Rightarrow F = B$; $A = 1 \Rightarrow F = C$.

2:1 MUX challenge: three-variable function (1/2)

FIND THE TWO COFACTORS BEFORE CONNECTING ANY WIRE

Target: $F(A, B, C) = \Sigma m(1, 2, 4, 5, 7)$ **Select:** $S = A$

Step 1: Group the truth table by A

BC	00	01	10	11
$F\|_{A=0}$	0	1	1	0
$F\|_{A=1}$	1	1	0	1

Step 2: Read the patterns

$0, 1, 1, 0 \Rightarrow B \oplus C$

$1, 1, 0, 1 \Rightarrow \overline{B} + C$

$$\boxed{D_0 = F|_{A=0} = B \oplus C, \quad D_1 = F|_{A=1} = \overline{B} + C}$$

2:1 MUX challenge: three-variable function (2/2)

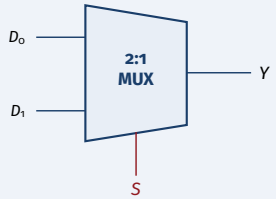
CONNECT THE DERIVED DATA FUNCTIONS AND VERIFY BOTH CASES

Target: $F = \Sigma m(1, 2, 4, 5, 7)$

MUX connections

Select	$S = A$
Input 0	$D_0 = B \oplus C$
Input 1	$D_1 = \overline{B} + C$
Output	$Y = F$

Extra gates: XOR for $B \oplus C$; NOT and OR for $\overline{B} + C$.



$D_0 \leftarrow B \oplus C$
$D_1 \leftarrow \overline{B} + C$
$S \leftarrow A$

$F = \overline{A}(B \oplus C) + A(\overline{B} + C)$ **Check:** $ABC = 110 \Rightarrow F = 0$; $ABC = 111 \Rightarrow F = 1$.

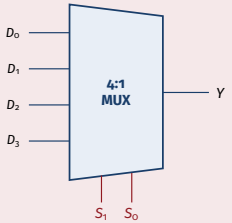
4:1 MUX example A: two-variable function

BOTH VARIABLES ARE SELECT INPUTS: ALL DATA PINS ARE CONSTANTS

Target: $F(A, B) = A \oplus B = \Sigma m(1, 2)$

Select: $S_1 = A,\ S_0 = B$

$S_1S_0 = AB$	F	Data connection
00	0	$D_0 = 0$
01	1	$D_1 = 1$
10	1	$D_2 = 1$
11	0	$D_3 = 0$



(D_0, D_1, D_2, D_3)
$= (0, 1, 1, 0)$

Comparison: The 2:1 implementation of XOR needed $\overline{B}$. The 4:1 implementation needs no data-function gates, but uses a larger MUX.

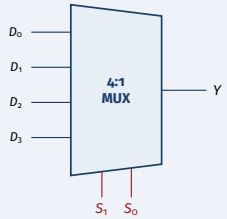
4:1 MUX example B: three-variable function

TWO VARIABLES SELECT THE PIN; ONE VARIABLE CONTROLS ITS DATA

Target: $F(A, B, C) = \Sigma m(1, 2, 6, 7)$ **Select:** $S_1 = A,\ S_0 = B$

AB	$F(C=0)$	$F(C=1)$	Connect
00	0	1	$D_0 = C$
01	1	0	$D_1 = \overline{C}$
10	0	0	$D_2 = 0$
11	1	1	$D_3 = 1$

$$\boxed{D_0 = C,\ D_1 = \overline{C},\ D_2 = 0,\ D_3 = 1}$$



$S_1 \leftarrow A$; $S_0 \leftarrow B$
Generate $\overline{C}$ with one NOT gate.

Verify: $ABC = 010 \Rightarrow D_1 = \overline{0} = 1$; $ABC = 101 \Rightarrow D_2 = 0$.

4:1 MUX example C: four-variable function (1/2)

EACH DATA PIN CAN BE A FUNCTION OF THE TWO REMAINING VARIABLES

Target: $F(A, B, C, D) = \Sigma m(1, 2, 4, 5, 9, 11, 13, 14, 15)$
Select: $S_1 = A$, $S_0 = B$; remaining variables are C, D.

Step 1: Compare the output for $CD = 00, 01, 10, 11$ in each AB group.

AB	$CD = 00$	01	10	11	Data function
00	0	1	1	0	$D_0 = C \oplus D$
01	1	1	0	0	$D_1 = \overline{C}$
10	0	1	0	1	$D_2 = D$
11	0	1	1	1	$D_3 = C + D$

Pattern recognition: 0110 is XOR; 1100 is $\overline{C}$; 0101 is D; 0111 is OR. The bit order matters.

4:1 MUX example C: four-variable function (2/2)

CONNECT EACH TWO-VARIABLE DATA FUNCTION TO THE CORRECT PIN

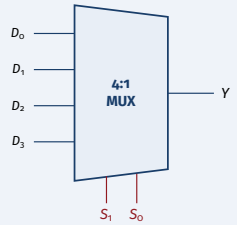
Select: $S_1 = A,\ S_0 = B$

$D_0 = C \oplus D$	XOR gate
$D_1 = \overline{C}$	NOT gate
$D_2 = D$	Direct wire
$D_3 = C + D$	OR gate

Two checks

$ABCD = 0110$: $AB = 01 \Rightarrow D_1 = \overline{C} = 0$.

$ABCD = 1110$: $AB = 11 \Rightarrow D_3 = C + D = 1$.



$Y = F(A, B, C, D)$

$$F = \overline{A}\overline{B}(C \oplus D) + \overline{A}B\,\overline{C} + A\overline{B}D + AB(C + D)$$

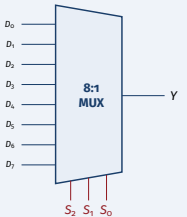
8:1 MUX example A: three-variable function

ALL THREE VARIABLES SELECT; DATA PINS NEED ONLY HIGH OR LOW

Target: $F(A, B, C) = \Sigma m(1, 2, 4, 7) = A \oplus B \oplus C$
Select: $S_2 = A,\ S_1 = B,\ S_0 = C$

ABC	000	001	010	011	100	101	110	111
Pin	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
Value	0	1	1	0	1	0	0	1

Connect: $D_1, D_2, D_4, D_7 \leftarrow 1$; $D_0, D_3, D_5, D_6 \leftarrow 0$.



$ABC = 010 \Rightarrow Y = D_2 = 1$
$ABC = 110 \Rightarrow Y = D_6 = 0$

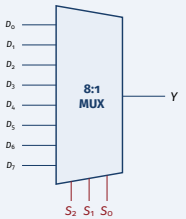
Connection to Lab 4: This is also the sum bit of a full adder.

8:1 MUX example B: four-variable function

ONE REMAINING VARIABLE: EACH DATA PIN IS 0, 1, D OR $\overline{D}$

Target: $F(A, B, C, D) = \Sigma m(1, 2, 6, 7, 8, 11, 12, 13)$ **Select:** $S_2 = A,\ S_1 = B,\ S_0 = C$

ABC	$F(D = 0)$	$F(D = 1)$	Connect
000	0	1	$D_0 = D$
001	1	0	$D_1 = \overline{D}$
010	0	0	$D_2 = 0$
011	1	1	$D_3 = 1$
100	1	0	$D_4 = \overline{D}$
101	0	1	$D_5 = D$
110	1	1	$D_6 = 1$
111	0	0	$D_7 = 0$



Extra gate: one inverter for $\overline{D}$.

$ABCD = 1000 \Rightarrow D_4 = \overline{D} = 1.$
$ABCD = 1010 \Rightarrow D_5 = D = 0.$

8:1 MUX example C: five-variable function (1/2)

EACH OF EIGHT DATA PINS NOW DEPENDS ON TWO VARIABLES

Target: $F(A, B, C, D, E) = \Sigma m(\ 1, 2, 4, 5, 11, 16, 17, 18, 19, 21, 22, 23, 24, 26, 30)$
Select: $S_2 = A$, $S_1 = B$, $S_0 = C$; vary remaining (D, E).

Step 1: For each fixed ABC, read the four outputs in DE order 00, 01, 10, 11.

ABC	$DE = 00$	01	10	11	Simplified data function
000	0	1	1	0	$D_0 = D \oplus E$
001	1	1	0	0	$D_1 = \overline{D}$
010	0	0	0	1	$D_2 = DE$
011	0	0	0	0	$D_3 = 0$
100	1	1	1	1	$D_4 = 1$
101	0	1	1	1	$D_5 = D + E$
110	1	0	1	0	$D_6 = \overline{E}$
111	0	0	1	0	$D_7 = D\overline{E}$

8:1 MUX example C: five-variable function (2/2)

BUILD THE INPUT FUNCTIONS, THEN LET THE MUX SELECT ONE

Select: $S_2 = A,\ S_1 = B,\ S_0 = C$

$D_0 = D \oplus E \qquad D_4 = 1$

$D_1 = \overline{D} \qquad D_5 = D + E$

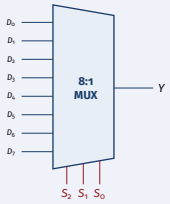
$D_2 = DE \qquad D_6 = \overline{E}$

$D_3 = 0 \qquad D_7 = D\overline{E}$

Verify two minterm cases

$ABCDE = 10111$: $ABC = 101$ and $DE = 11$, so $D_5 = D + E = 1$ (minterm 23).

$ABCDE = 11111$: $ABC = 111$ and $DE = 11$, so $D_7 = D\overline{E} = 0$ (not in the list).



Gate generators: XOR, NOT, AND, OR.
Reuse $\overline{E}$ for D_6 and D_7.

Important: An 8:1 MUX can implement a five-variable function, but its data inputs may require additional combinational logic.

What changes when the function has more variables?

THE INPUT COUNT OF A MUX DOES NOT LIMIT THE NUMBER OF VARIABLES IN A FUNCTION

MUX	Select variables	Variables left	What drives data pins?
2:1	1	0 / 1 / 2	Constants, signals, or 2-variable logic
4:1	2	0 / 1 / 2	Constants, signals, or 2-variable logic
8:1	3	0 / 1 / 2	Constants, signals, or 2-variable logic

If every variable is a select input
Each D_i is a constant 0 or 1.

If variables remain
Each D_i is a function of those variables.

Design checklist: choose select bits $\rightarrow$ group truth-table rows $\rightarrow$ simplify each D_i $\rightarrow$ wire extra gates $\rightarrow$ verify.

A repeatable five-step MUX design method

USE THIS FOR EVERY BOOLEAN-IMPLEMENTATION QUESTION

1. Write the function's truth table or minterms.
2. Choose select variables and their bit order.
3. For each selection, compute the required output.
4. Connect data inputs to 0, 1, a variable, its complement, or subfunction.
5. Verify *every* truth-table row.

Try on your own

Using a 4:1 MUX, implement:

$F(A, B, C) = \Sigma m(0, 3, 5, 6)$

Choose $S_1 = A$, $S_0 = B$.

Self-check answer

$D_0 = \overline{C}$; $D_1 = C$; $D_2 = C$; $D_3 = \overline{C}$.

The SN74LS153: two 4:1 MUX sections in one IC

A REAL LABORATORY IMPLEMENTATION

- **Dual 4-to-1 selector**: two channels.
- **Shared select bits**: B (MSB), A (LSB).
- **Separate active-low enables**: $\overline{1G}$ and $\overline{2G}$.
- **Independent outputs**: $1Y$ and $2Y$.
- **Normal 5 V TTL supply**: connect V_{CC} and GND correctly.

Section 1

$$1Y = 1C_{BA} \text{ if } 1G = 0$$

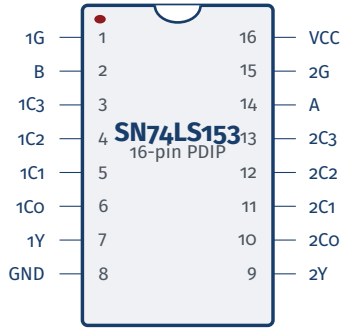
$$1Y = 0 \text{ if } 1G = 1$$

C_{BA} means the input indexed by binary BA.

Use section 1 for the experiments; keep the unused section's enable inactive.

Source: Texas Instruments, *SN74LS153 Dual 4-Line to 1-Line Data Selectors/Multiplexers* datasheet.

SN74LS153: manufacturer-verified pinout (top view)



Section 1 connections

Pin 1	$\overline{1G}$: enable
Pin 2	$B = S_1$
Pin 14	$A = S_0$
Pin 6	$D_0 = 1C0$
Pin 5	$D_1 = 1C1$
Pin 4	$D_2 = 1C2$
Pin 3	$D_3 = 1C3$
Pin 7	$Y = 1Y$
Pin 16	$V_{CC} = +5$ V
Pin 8	GND

View from above, with the notch at top. Never interpret a diagram as a bottom view.

Source: TI SN74LS153 datasheet, 16-pin D/N package, top-view pin assignment.

SN74LS153 function table and switching behavior

$1\overline{G}$	B	A	$1Y$
1	X	X	0
0	0	0	$1C0$
0	0	1	$1C1$
0	1	0	$1C2$
0	1	1	$1C3$

X: don't care; the output does not depend on that input in that row.

Do not assume instant switching

Real logic ICs have *propagation delay*: output changes occur a short time after input changes.

Avoid ambiguous input states

Unconnected TTL inputs are not a valid substitute for carefully defined logic levels.

Key idea *B* is the most significant select bit; *A* is the least significant select bit.

Before wiring: a precise connection map

SN74LS153 — USE SECTION 1

Function	Pin	Connect to
Power	16	regulated +5 V
Ground	8	common GND
Enable 1$\overline{G}$	1	logic 0 (active)
Select B	2	switch S_1
Select A	14	switch S_0
Input 1C0	6	data switch D_0
Input 1C1	5	data switch D_1
Input 1C2	4	data switch D_2
Input 1C3	3	data switch D_3
Output 1Y	7	logic indicator
Unused 2$\overline{G}$	15	logic 1 (disable)

Sequence

1. Disconnect power.
2. Locate notch and pin 1.
3. Connect supply and ground.
4. Connect select and data switches.
5. Connect output to a logic indicator.
6. Check wiring, then power on.

Use a 0.1 μF bypass capacitor near the IC supply pins where practicable. Never connect an output directly to a supply rail.

How to generate $\overline{C}$: the 74LS04 inverter

ADDITIONAL IC FOR LAB EXPERIMENT 3

Use one gate from a 74LS04 (14-pin PDIP)

Pin	Connection
14	regulated +5 V
7	common GND
1 (1A)	variable C
2 (1Y)	$\overline{C}$ (to SN74LS153 pin 5)

Tie unused inverter *inputs* to defined logic levels; leave their outputs unconnected. Observe the exact package and datasheet.



74LS04 pins 1 → 2

C	$\overline{C}$
0	1
1	0

Source: Texas Instruments, SN74LS04 Hex Inverters datasheet, D/N package (top view).

Lab experiment 1: verify a 4:1 MUX

OBJECTIVE: PROVE THAT ONLY THE SELECTED INPUT REACHES THE OUTPUT

Set these data levels

$(D_0, D_1, D_2, D_3) = (0, 1, 1, 0)$

Connect $1G = 0$ to enable section 1.

Observe all four select states

BA	Selected	Expected Y
00	D_0	0
01	D_1	1
10	D_2	1
11	D_3	0

Second test: data follows selection

Hold $BA = 10$; toggle D_2 from 0 to 1. The output must follow D_2.

Third test: unselected input

Keep $BA = 10$; toggle D_0. The output should not change.

Lab experiment 2: verify enable control

$1G$	BA	Selected level	Expected $1Y$
0	01	$D_1 = 1$	1
1	01	$D_1 = 1$	0
0	01	$D_1 = 1$	1

Change only the enable signal.
Keep select bits and data constant during the test.

Why this is useful

It demonstrates the difference between *selecting* an input and *enabling* a MUX section.

Report result

When $1G = 1$, the SN74LS153 output is forced LOW.

Lab experiment 3: implement a Boolean function

$F(A, B, C) = \Sigma m(1, 2, 6, 7)$

Connect select lines

$B \leftarrow A$, $A_{\text{pin}} \leftarrow B_{\text{variable}}$

Connect data inputs

1C0 $\leftarrow C$
1C1 $\leftarrow \overline{C}$
1C2 $\leftarrow$ 0
1C3 $\leftarrow$ 1

The IC pin is named **A**, but the Boolean variable is also named *A*. Follow the arrows to avoid mixing the two.

Verify all 8 input rows

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Observation sheet: expected vs. measured output

RECORD EVIDENCE, NOT ONLY THE FINAL ANSWER

Input variables			Select *BA*	Expected	Observed	Match?
A	*B*	*C*	*AB*	*F*	*Y*	Yes/No
0	0	0	00	0	_____	____
0	0	1	00	1	_____	____
0	1	0	01	1	_____	____
0	1	1	01	0	_____	____
1	0	0	10	0	_____	____
1	0	1	10	0	_____	____
1	1	0	11	1	_____	____
1	1	1	11	1	_____	____

If an observation differs, note the **input state**, **wrong value**, and **likely cause**, then correct and retest.

Troubleshooting and safe hardware practice

Symptom → possible cause

- Always LOW → enable HIGH, output wiring.
- Wrong D_i selected → A/B swapped.
- Unstable reading → floating switch/input.
- No response → power, GND, orientation.
- Wrong complement → inverter connection.

Safety / reliability checklist

- Turn power OFF before rewiring.
- Check VCC and GND before inserting the IC.
- Define all used digital input levels.
- Do not short outputs to power or ground.
- Use proper logic probe or buffered LED.
- Verify IC family and its datasheet.

What is a multiplexer used for?

BEYOND THE LABORATORY



Datapath selection

Choose one register, sensor, or input source for a shared path.



Logic implementation

Implement combinational truth tables without a dedicated gate network.



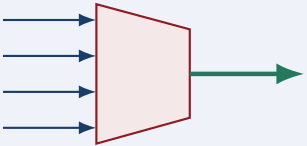
Shared channels

Select among multiple digital sources for one receiver.

Key idea A MUX is a decision element in digital *data paths*, not a data storage element.

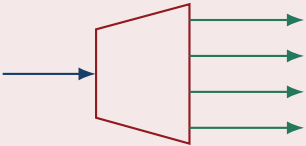
MUX now; DEMUX in the next laboratory

Multiplexer



Multiple inputs → one output.

Demultiplexer



One input → selected output.

Key idea Both use select lines, but the direction of selection differs. DEMUX is covered in Lab 7.

Viva and exit check

CAN YOU EXPLAIN YOUR CIRCUIT, NOT JUST ITS OUTPUT?

Fundamentals

1. Why does a 4:1 MUX use two selects?
2. What does $S_1S_0 = 11$ mean?
3. Derive the 2:1 MUX equation.
4. Is the MUX combinational or sequential?

Design and lab

5. How do you implement $\Sigma m(1, 2, 6, 7)$ using 4:1?
6. What happens when $1G = 1$ on SN74LS153?
7. Which pin is VCC? Which is GND?
8. Why test unselected inputs?

Exit task: Draw a 4:1 block, annotate the pins, and explain Y for select 10.

Summary: four ideas you must remember

1. Selection
n select lines can address 2^n inputs.

2. Boolean model
$Y = \overline{S}D_0 + SD_1$ for a 2:1 MUX.

3. Logic synthesis
Data inputs can implement truth-table values or functions.

4. Hardware verification
Check pinout, enable, select inputs, power, and every test case.

Next laboratory: Demultiplexer (DEMUX)

References and laboratory source notes

Presidency University (CSE Department). *CSE208: Digital Logic Design Laboratory Course Outline*. Lab 6: Multiplexer; CO3.

Mano, M. M., and Ciletti, M. D. (2018). *Digital Design*, 6th ed. Pearson. Combinational logic and multiplexers.

Tocci, R. J., Widmer, N. S., and Moss, G. L. (2014). *Digital Systems: Principles and Applications*, 11th ed. Pearson.

Texas Instruments. *SN74LS153 Dual 4-Line to 1-Line Data Selectors/Multiplexers*, datasheet (Rev. A); *SN74LS04 Hex Inverters*, datasheet (Rev. C).
https://www.ti.com/product/SN74LS153